

# Data Mining

Complete Course Notes — Explained

---

| Topic  | Content                                      |
|--------|----------------------------------------------|
| Part 1 | Association Rules — Apriori Algorithm        |
| Part 2 | Quality Measures — Support, Confidence, Lift |
| Part 3 | Closed Itemsets Algorithm                    |
| Part 4 | Clustering — Concepts & Approaches           |
| Part 5 | K-Means Algorithm                            |
| Part 6 | Hierarchical Clustering (CAH)                |
| Part 7 | DBSCAN Algorithm                             |

## PART 1 — Association Rules (Règles d'Association)

### What is Data Mining?

Data Mining is a process used to **analyze very large quantities of data** in order to discover useful, hidden, or important information. The goal is to **transform raw data into useful information and knowledge** — patterns and facts that are not obvious by simply looking at the data.

### What does Data Mining look for?

- **Repetition** — Patterns that occur frequently in the data.
- **Association** — Items that are often found together (e.g., coffee + sugar).
- **Frequent Behaviors** — Common sequences or actions by users/customers.

### Key Concepts

#### Association

Two products are often bought together. Example: *coffee and sugar* — customers who buy coffee tend to also buy sugar.

#### Correlation

Two things evolve together simultaneously. Example: *temperature*  $\uparrow \rightarrow$  *ice cream sales*  $\uparrow$ . When temperature rises, ice cream sales also rise.

#### Basket Analysis (Analyse du Panier)

A method to study what customers buy together in the same shopping basket. Supermarkets use this to organize shelves and create promotions.

### Two-Step Approach

Mining association rules is done in two major steps:

- **Step 1 — Extraction of Frequent Itemsets:** Find all combinations of items that appear together often enough.
- **Step 2 — Rule Generation:** From frequent itemsets, generate association rules of the form  $A \rightarrow B$ .

### Core Definitions

| Term             | Definition                            | Example                                            |
|------------------|---------------------------------------|----------------------------------------------------|
| Itemset          | A collection (set) of items           | {coffee, sugar, milk}                              |
| Transaction      | A purchase, sale, or any single event | T1: customer buys {A, C, E}                        |
| Association Rule | 'If A then B' — A implies B           | $A \rightarrow B$ (if a client buys A, they buy B) |
| Antecedent (A)   | The left-hand side of the rule        | A (the condition)                                  |
| Consequent (B)   | The right-hand side of the rule       | B (the result)                                     |

## PART 2 — Quality Measures: Support, Confidence & Lift

### 1. Support

The **support** of an itemset  $X$  is the **proportion of all transactions** that contain  $X$ . It measures how frequently  $X$  appears in the dataset.

$$\text{Sup}(X) = \text{Freq}(X) / |D|$$

Where:

- **Freq(X)** = number of transactions containing  $X$
- **|D|** = total number of transactions in the dataset

#### Frequent Itemset

An itemset  $X$  is called **frequent** if its support is greater than or equal to a minimum threshold (minsup) set by the user:

$$\text{Sup}(X) \geq \text{minsup} \quad (\text{minsup} \in [0, 1])$$

■ minsup is chosen by the user, typically between 0 and 1 (e.g., 0.4 means the itemset must appear in at least 40% of transactions).

### 2. Monotonicity Properties

#### Monotone Property (Anti-monotonicity of infrequency)

If an itemset is **frequent**, then **all its subsets are also frequent**.

■ Example: If  $\{A,B,C,D\}$  is frequent  $\rightarrow \{ABC\}, \{ABD\}, \{ACD\}, \{BCD\}, \{AB\}, \{AC\}, \dots$  are ALL frequent.

#### Anti-Monotone Property (Pruning)

If an itemset is **infrequent**, then **all its supersets are also infrequent**. This is called **pruning (élagage)** — we can skip testing supersets of infrequent itemsets.

■ Example: If  $\{A,B\}$  is infrequent  $\rightarrow \{ABC\}, \{ABD\}, \{ABCD\}$  are all infrequent automatically.

### 3. Confidence

The **confidence** of a rule  $A \rightarrow B$  is the **proportion of transactions containing A that also contain B**. It measures how reliable the rule is.

$$\text{Confidence}(A \rightarrow B) = \text{Sup}(AB) / \text{Sup}(A)$$

- **Solid Rule:**  $\text{Confidence}(A \rightarrow B) \geq \text{MinConf}$  (user-defined minimum confidence threshold)
- **Strong Rule:**  $\text{Confidence}(A \rightarrow B) = 1$  (100%) — the rule is always true. Every transaction with  $A$  also contains  $B$ .

### 4. Lift

The **lift** measures whether two elements are truly linked or just appear together by chance. It compares the observed co-occurrence vs what would be expected if they were independent.

$$\text{Lift}(A \rightarrow B) = \text{Sup}(A \cap B) / (\text{Sup}(A) \times \text{Sup}(B))$$

Also written as:

$$\text{Lift}(A \rightarrow B) = \text{Conf}(A \rightarrow B) / \text{Sup}(B)$$

| Lift Value | Interpretation                                                                  |
|------------|---------------------------------------------------------------------------------|
| Lift > 1   | Positive correlation — A and B appear TOGETHER more often than by chance        |
| Lift = 1   | No relationship — A and B are independent                                       |
| Lift < 1   | Negative correlation — A and B tend to AVOID each other (never bought together) |

## PART 3 — Apriori Algorithm

The **Apriori algorithm** is the classic method for extracting frequent itemsets from a dataset. It uses the anti-monotone property to prune the search space efficiently.

### Inputs & Outputs

| Input                                             | Output                       |
|---------------------------------------------------|------------------------------|
| Extraction context K (the dataset / transactions) | Set of all frequent itemsets |
| MinSup (minimum support threshold)                |                              |

### Algorithm Steps

- **1 — Initialization:** Create all candidate itemsets of size 1 (C1). Every single item becomes a candidate.
- **2 — Main Loop:** Calculate the support of each candidate itemset by scanning all transactions.
- **3 — Pruning (Élagage):** Remove all candidates whose support < MinSup. They are infrequent.
- **4 — Generate New Candidates:** Combine frequent itemsets of size k to generate candidates of size k+1.
- **Repeat:** Loop back to step 2 until no more candidates can be generated.

### Worked Example (MinSup = $2/5 = 0.4$ )

Dataset with 5 transactions (T1–T5) and 5 items (A, B, C, D, E):

|    | A | B | C | D | E |
|----|---|---|---|---|---|
| T1 | 1 | 0 | 1 | 0 | 1 |
| T2 | 1 | 1 | 1 | 0 | 0 |
| T3 | 1 | 0 | 0 | 1 | 1 |
| T4 | 0 | 1 | 1 | 0 | 1 |
| T5 | 0 | 1 | 1 | 0 | 0 |

#### Step 1 — Itemsets of size 1 (Candidates C1)

| Item | Support     | Frequent?     |
|------|-------------|---------------|
| A    | $3/5 = 0.6$ | ✓ Yes         |
| B    | $3/5 = 0.6$ | ✓ Yes         |
| C    | $4/5 = 0.8$ | ✓ Yes         |
| D    | $1/5 = 0.2$ | ✗ No (pruned) |
| E    | $3/5 = 0.6$ | ✓ Yes         |

■ D is pruned (support  $0.2 < 0.4$ ). All supersets of D are also discarded.

#### Step 2 — Itemsets of size 2 (Candidates C2)

We combine frequent items {A, B, C, E} pairwise:

| Itemset | Support     | Frequent? |
|---------|-------------|-----------|
| AB      | $1/5 = 0.2$ | ✗ No      |
| AC      | $2/5 = 0.4$ | ✓ Yes     |
| AE      | $2/5 = 0.4$ | ✓ Yes     |
| BC      | $3/5 = 0.6$ | ✓ Yes     |
| BE      | $0/5 = 0.0$ | ✗ No      |
| CE      | $2/5 = 0.4$ | ✓ Yes     |

### Step 3 — Itemsets of size 3 (Candidates C3)

Frequent 2-itemsets are {AC, AE, BC, CE}. To form 3-itemsets, two itemsets must share a common prefix.

AC + AE → ACE (common prefix: A)

| Itemset | Support     | Frequent?     |
|---------|-------------|---------------|
| ACE     | $1/5 = 0.2$ | ✗ No (pruned) |

■ Result: The frequent itemsets are — A, B, C, E, AC, AE, BC, CE.

## PART 4 — Closed Itemsets Algorithm (Algorithme Close)

The **Closed Itemsets** algorithm is an optimized approach to find frequent itemsets. An itemset  $X$  is called **closed** if none of its supersets has the same support as  $X$ . This avoids storing redundant information.

### Key Idea

When moving from  $k=1$  to  $k=2$ , then  $k=3$ , we don't keep itemsets that are already **represented by a closure** (a superset with the same support). This dramatically reduces the number of itemsets we need to store.

### How Closure Works

For each itemset, we compute its **closure** = the largest set of items that always appears together with it across all transactions.

Example: If in the transaction table, item  $A$  and items  $C$ ,  $D$  always appear together, then the closure of  $A$  includes  $C$  and  $D$ . Once we store the closure, we don't need to separately store  $A$ ,  $AC$ ,  $AD$  — they're all represented.

■ This is the basis of algorithms like CHARM and CLOSET, which are more memory-efficient than Apriori for large datasets.

## PART 5 — Clustering

### What is Clustering?

A **cluster** is a group of similar objects. **Clustering** is a method that **automatically creates groups of similar objects** from data, without any predefined labels. It is an **unsupervised** learning technique.

Clustering is a **complex problem** because there are an enormous number of possible ways to group objects. The algorithm must find the best partition.

### Two Scenarios for K (number of clusters)

| K is Fixed (known)                                                       | K is Not Fixed (unknown)                                                          |
|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| The algorithm searches only for the best partition into exactly K groups | The algorithm must test all possible numbers of clusters: $K = 1, 2, 3, \dots, N$ |
| Faster and more efficient                                                | Much more computationally expensive                                               |
| Example: K-Means                                                         | Requires trying many values of K                                                  |

### Objectives of Clustering

- **Maximize intra-cluster similarity:** Objects within the same cluster should be as similar (close) as possible.
- **Minimize inter-cluster similarity:** Objects in different clusters should be as different (distant) as possible.

### Types of Clustering

| Type             | Description                                                     | Example               |
|------------------|-----------------------------------------------------------------|-----------------------|
| Hard Clustering  | Each object belongs to exactly ONE cluster                      | K-Means               |
| Soft Clustering  | An object can belong to MULTIPLE clusters                       | Fuzzy C-Means         |
| Fuzzy Clustering | Each object belongs to ALL clusters with a degree of membership | 90% C1, 10% C2, 0% C3 |



## PART 6 — Distance Calculation

To measure **similarity** between objects, we use **distance metrics**. A small distance means objects are similar; a large distance means they are different.

### Properties of Distance

- **Symmetry:**  $d(x_i, x_j) = d(x_j, x_i)$
- **Positivity:**  $d \geq 0$
- **Identity:**  $d(x_i, x_j) = 0$  if and only if  $x_i = x_j$  (same object)

### Types of Distance

#### 1. Euclidean Distance (for numerical data)

$$D(x_i, x_j) = \sqrt{\sum_k (x_{ik} - x_{jk})^2}$$

The straight-line distance between two points in space.

■ Example:  $D(x_1, x_2)$  where  $x_1=(0,2)$  and  $x_2=(0,0) \rightarrow D = \sqrt{(0-0)^2 + (2-0)^2} = 2$

#### 2. Manhattan Distance (City Block)

$$D(x_i, x_j) = \sum_k |x_{ik} - x_{jk}|$$

The sum of absolute differences — like navigating city blocks.

#### 3. Hamming Distance (for binary data)

Number of positions where two binary strings differ. Used when data is 0/1 (present/absent).

#### 4. Categorical Distance

For non-numeric values (categories), comparison is: equal  $\rightarrow$  distance=0, different  $\rightarrow$  distance=1.

$$D_c(x_i, x_j) = \sum_k f(x_{ik}, x_{jk})$$

### Total Distance

For datasets with both numerical and categorical attributes:

$$D_{\text{total}} = D_{\text{numerical}} + D_{\text{categorical}}$$

### Cluster Quality Evaluation

| Measure                | Goal                       | Interpretation                     |
|------------------------|----------------------------|------------------------------------|
| Inter-cluster distance | Between different clusters | Larger = better separated clusters |
| Intra-cluster distance | Within the same cluster    | Smaller = more compact clusters    |

### Inter-cluster Distance Methods

- **Single Link:** Take the SMALLEST distance between two clusters (closest pair of points).
- **Complete Link:** Take the LARGEST distance between two clusters (farthest pair of points).
- **Average Link:** Take the AVERAGE of all pairwise distances between the two clusters.

- **Centroid Link:** Calculate one center point per cluster, then measure distance between centers.
- **Mean Centroid Link:** Average of the two cluster centers.

## PART 7 — K-Means Algorithm

K-Means is a **partitional clustering algorithm**. It divides data into exactly K clusters. Its complexity is  $O(t \times k \times N)$  where  $t$  = iterations,  $k$  = clusters,  $N$  = data points.

### K-Means Steps

| Step | Action                                                                        |
|------|-------------------------------------------------------------------------------|
| 1    | Choose K (number of clusters)                                                 |
| 2    | Place K initial centers (centroids) RANDOMLY in the data space                |
| 3    | Assign each data point to the NEAREST center (using distance metric)          |
| 4    | Recalculate the center of each cluster (mean of all points in the cluster)    |
| 5    | Repeat steps 3 and 4 until the clusters are STABLE (no point changes cluster) |

### Data Normalization

A critical preprocessing step! Without normalization, attributes with **large values will dominate** the distance calculation, making attributes with small values irrelevant.

The idea: make all attributes **equally balanced** on the same scale.

#### Mean Absolute Deviation

$$S_g = (1/n) \times (|x_{1g} - m_g| + |x_{2g} - m_g| + \dots + |x_{ng} - m_g|)$$

Where  $m_g = (1/n)(x_1 + x_2 + \dots + x_n)$  is the mean of attribute  $g$ .

#### Standardized Measure (Z-score)

$$Z_{ig} = (x_{ig} - m_g) / S_g$$

After normalization, all values are on the same scale (typically mean=0, std=1).

## PART 8 — Hierarchical Clustering (Clustering Hiérarchique)

Hierarchical clustering builds a **tree-like structure (dendrogram)** showing how clusters are formed step by step. Unlike K-Means, you do **NOT need to specify K in advance**.

### Two Types

#### 1. Agglomerative (Bottom-Up) — CAH

Start with each object as its own cluster, then **merge** the two closest clusters repeatedly until all objects are in one cluster.

#### 2. Divisive (Top-Down)

Start with all objects in one big cluster, then **split** clusters repeatedly until each object is its own cluster.

### CAH (Agglomerative Hierarchical Clustering) — Steps

- **Step 1:** Each object starts as its own cluster.
- **Step 2:** Calculate distances between ALL pairs of clusters.
- **Step 3:** Merge the two clusters with the SMALLEST distance.
- **Step 4:** Recalculate the center of the new merged cluster:  $C = (1/n) \times \sum(x_i)$
- **Step 5:** Repeat until only one cluster remains.

### Dendrogram

A **dendrogram** is a tree diagram that shows how clusters merge at each step and at what distance. By cutting the dendrogram at a chosen height, you can get any number of clusters.

■ The horizontal axis shows the objects, and the vertical axis shows the distance at which clusters were merged. Cutting higher → fewer clusters; cutting lower → more clusters.

## PART 9 — DBSCAN Algorithm

DBSCAN (**D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise) is a clustering algorithm that groups points based on **density**. Unlike K-Means, it can find clusters of any shape and can identify **noise** (outliers).

### Parameters

| Parameter             | Meaning                                                             |
|-----------------------|---------------------------------------------------------------------|
| R (Radius)            | The maximum distance to consider two points as neighbors            |
| MinP (Minimum Points) | The minimum number of neighbors required to consider a region dense |

### Types of Points in DBSCAN

| Point Type                   | Definition                                                            |
|------------------------------|-----------------------------------------------------------------------|
| Core Point (Point Central)   | Has AT LEAST MinP neighbors within radius R. It is in a dense region. |
| Border Point (Point Bordure) | Has FEWER than MinP neighbors, but is CLOSE to a core point.          |
| Noise Point (Bruit)          | Isolated — not close to any core point. It is an outlier.             |

### How DBSCAN Works

- Pick an unvisited point. If it has  $\geq$  MinP neighbors within radius R  $\rightarrow$  it's a CORE point, start a new cluster.
- Add all neighbors of this core point to the cluster.
- For each neighbor that is also a core point, add THEIR neighbors too (expand the cluster).
- Continue until no more points can be added. Border points are added but don't expand.
- Points that don't belong to any cluster are labeled as NOISE.

### DBSCAN vs K-Means Comparison

| Feature            | K-Means                          | DBSCAN                         |
|--------------------|----------------------------------|--------------------------------|
| Number of clusters | Must specify K in advance        | Discovered automatically       |
| Cluster shape      | Only spherical/convex shapes     | Any shape                      |
| Noise handling     | All points assigned to a cluster | Noise points identified        |
| Parameters         | K (number of clusters)           | R (radius) + MinP (min points) |
| Sensitivity        | Sensitive to initial centers     | Sensitive to R and MinP values |